

Description of our Approach

Jakub Jęsień, Jakub Radziejewski

Architecture and Methodology

This system implements a weakly-supervised crack segmentation approach through a two-stage pipeline. The architecture addresses the challenge of segmenting cracks when only image-level labels (crack/no-crack) are available during training, not pixel-level masks.

Overview Flow

Input Image (448×448)



Classification + Grad-CAM++



Confidence Score + Attention Map



Pseudo-label Generation



Binary Pseudo-mask



U-Net Segmentation



Refined Segmentation Mask

Classification with Grad-CAM++

The first stage uses a modified ResNet-34 architecture which was pretrained on ImageNet that performs binary classification while generating spatial attention maps through Grad-CAM++. The network processes 448×448 pixel input images through a series of convolutional and residual blocks.

ResNet-34 Flow

The baseline architecture follows this processing pipeline:

Input (448×448)



Conv1 + BN + ReLU + MaxPool



Layer 1 (56×56)



Layer 2 (28×28)



Layer 3 (28×28) - stride 1, dilation 2



Layer 4 (28×28) - stride 1, dilation 4



Global Average Pooling



Fully Connected Layer



Output 1 - Classification: Crack probability

Output 2 - Attention Maps: From layers 2, 3, 4 (all at 28×28)

Grad-CAM++ Mechanism

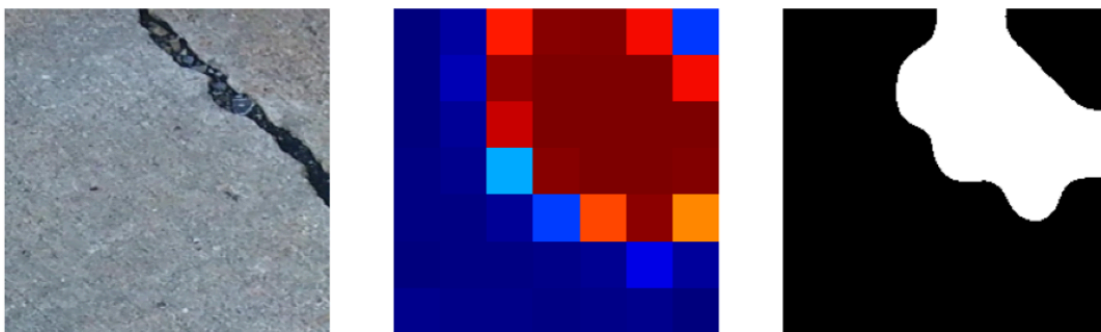
During forward passes intended for attention map generation, the network registers gradient hooks on selected residual layers (2,3,4) to capture both gradients and activations. After computing the classification output, we backpropagate the class score for the "crack" category. The Grad-CAM++ algorithm then analyzes these gradients and activations to determine which spatial locations contributed most to the classification decision.

Specifically, Grad-CAM++ computes pixel-wise weighting coefficients (α) by analyzing second and third order derivatives of the class score with respect to feature map activations. These coefficients are combined with positive gradients and summed across channels to produce a spatial attention map. This map highlights regions the network considers most relevant for crack detection.

Problem we had when designing classifier part

Initial experiments were performed with standard ResNet-18. By the time features reach layer four, the spatial resolution was reduced to only 7×7 pixels. When Grad-CAM++ generated attention maps from this resolution, the results were disappointingly vague and lacked the spatial detail needed for meaningful crack localization.

The attention maps correctly showed general regions of interest but were sometimes a blobs of size quarter or even half of the picture. Upsampling these 7×7 maps to the target 448×448 resolution simply magnified the coarseness rather than revealing useful spatial structure. The fundamental problem was that too much spatial information had been discarded through aggressive downsampling.



Here an example 7×7 activation map (in the middle), which correctly localises crack place but even with UNet refinement, final mask produced (on the right) is far away from being close to original crack image (on the left)

First Solution: Multi-scale Fusion

To address the vagueness problem, we first implemented multi-scale attention fusion. Rather than relying solely on layer four's coarse 7×7 activations, we extracted Grad-CAM++ attention maps from three different residual layers: layer two, layer three, and layer four.

Each layer captures different information. Layer two operates at higher spatial resolution (28×28) and detects fine-grained patterns like crack edges and textures. Layer three (14×14) captures mid-level features and partial crack structures. Layer four (7×7) encodes high-level semantic understanding of overall crack presence.

By combining these three perspectives with weights at that time - $[0.5, 0.3, 0.2]$, the fused attention maps became noticeably more detailed and spatially informative than using layer four alone. Final mask localized cracks much more effectively, improved our results by approximately 0.05 on dice but still we felt that there is a potential to improve.

Final Solution: Dilated Convolutions

We implemented a second architectural modification targeting layers three and four. Standard ResNet architectures either 18 or 34 use stride-2 convolutions in these layers to reduce spatial dimensions. We manually modified these layers to use stride-1 convolutions instead, preventing further downsampling after layer two.

However, simply removing strides would improve the performance, that's why we introduced dilated convolutions. Layer three uses $\text{dilation}=2$, and layer four uses $\text{dilation}=4$. This configuration maintains the same effective receptive field as the original architecture while preserving the 28×28 spatial resolution throughout all deeper layers.

For each convolutional layer in layers three and four, we changed stride from (2,2) to (1,1) and adjusted the dilation parameter accordingly. The padding was also modified to match the dilation ($\text{padding} = \text{dilation}$ for 3×3 kernels) to maintain consistent spatial dimensions.

After we came up with a better solution, we decided to try ResNet 34 which improved dice by around 0.03 by just running one epoch. Our final solution was tested on 2 epochs and the dice result went up overall by around 0.06. It's highly likely that the score will be further improved with more epochs.

Confidence Threshold Selection

After training the classifier, we evaluate it on the validation set to collect predicted crack probabilities and ground-truth labels. Using scikit-learn's *precision_recall_curve*, we compute F1 scores across all possible thresholds and select the value that maximizes F1 score. This ensures a balance between precision and recall.

Pseudo-label Generation

The improved attention maps are converted to binary pseudo-masks through a two-step filtering process that ensures quality control.

For each training image:

↓

Classify → Confidence Score

↓

If confidence < threshold:

→ Assign EMPTY mask (all zeros)

→ Skip Grad-CAM++ computation

Else:

→ Generate multi-scale attention map

Extract Layer 2 CAM (fine details)

Extract Layer 3 CAM (mid-level)

Extract Layer 4 CAM (semantic)

→ Fuse with weights (0.2, 0.4, 0.4)

→ Apply percentile threshold (95th)

→ Create binary pseudo-mask 1 if fused_cam > threshold

The confidence threshold acts as a first-line filter. Images where the classifier is uncertain (confidence below threshold) receive empty masks, preventing false positives from contaminating the training data. This threshold is automatically determined by maximizing F1 score on the validation set.

For images exceeding the confidence threshold, we generate the fused Grad-CAM++ attention map and apply percentile-based thresholding. Only the top 5% most activated regions (95th percentile) are marked as crack pixels. This conservative approach ensures

pseudo-labels capture only the most reliable, strongly activated crack regions while avoiding uncertain areas that might introduce noise.

Steps:

1. **Confidence Filtering:** Images with crack confidence $<$ threshold (determined by F1 optimization on validation set) receive empty masks
2. **Multi-scale CAM Extraction:** Generate GradCAM++ from layers 2, 3, and 4
3. **Fusion:** Weighted average ($0.2 \times \text{layer2} + 0.4 \times \text{layer3} + 0.4 \times \text{layer4}$)
4. **Binarization:** Apply percentile thresholding (default: 95th percentile)

Output: Pseudo-masks (448×448 binary masks) for all training images

Segmentation Stage - Lightweight U-Net

Why U-Net?

Even with our improvements, Grad-CAM++ attention maps remained rough. At 28×28 resolution, upsampling by 16× creates blob-like boundaries. Grad-CAM++ only highlights the most discriminative crack regions needed for classification, it doesn't focus on specific pixels.

Our idea to overcome it was to use U-Net.

U-Net takes RGB images as input and is trained to match the pseudo-mask targets. Through this process, it learns from both sources: the RGB images teach it what cracks look like visually (textures, edges, lighting), while the pseudo-masks guide it toward which regions to segment.

For example, when Grad-CAM++ highlights only crack centers, U-Net learns to recognize crack edges in the RGB image and extends masks to include them.

Training Setup

Input (x): RGB images (448×448)

Target (y): Pseudo-masks from Grad-CAM++

Output: Pixel-wise probability maps (448×448), each pixel $\in [0,1]$ represents crack probability

Loss: Binary Cross-Entropy between predicted probabilities and pseudo-mask labels

Data: All training images with their pseudo-labels (no validation - pseudo-labels are already noisy supervision)

Segmentation Phase Flow:

Input Image (448×448)

↓

Encoder Block 1 (Conv→BN→ReLU×2) → $448 \times 448 \times 32$ ↵

↓ MaxPool |

Encoder Block 2 (Conv→BN→ReLU×2) → $224 \times 224 \times 64$ ↵

↓ MaxPool |

Encoder Block 3 (Conv→BN→ReLU×2) → $112 \times 112 \times 128$ ↵

↓ MaxPool |

Bottleneck (Conv→BN→ReLU×2) → $56 \times 56 \times 256$ |

↓ Upsample |

Decoder Block 3 ←————— (Concatenate)

↓ Upsample

Decoder Block 2 ←————— (Concatenate)

↓ Upsample

Decoder Block 1 ←————— (Concatenate)

↓

Output Conv (1×1) → Sigmoid → Probability Map (448×448)

Skip connections concatenate encoder features to decoder layers, preserving spatial detail while combining it with semantic information for precise boundaries

What U-Net Learns

U-Net overcomes pseudo-label limitations by learning:

- Extends partial activations into full, connected crack structures
- Converts fuzzy 28×28 attention into precise 448×448 pixel edges
- Filters false activations on textured backgrounds

- Enforces spatial coherence—cracks are continuous with consistent widths

Essentially, the U-Net acts as a learned post-processing and enhancement filter.

When we tested our almost final version without U-Net, the Dice score dropped by around 0.10.

Test set metrics:		Test set metrics:	
Mean IoU:	0.2163	Mean IoU:	0.3064
Mean Dice:	0.2835	Mean Dice:	0.4015
Precision:	0.0936	Precision:	0.2952
Recall:	0.7624	Recall:	0.3607

On the left results without UNet, and on the right with UNet, both runs were made on only one epoch, due to computation time and our limitation regarding computational power.

Otsu Threshold

Input: Segmentation model predicted masks (sample size: 1000 masks, as it already gave good enough results, when run all on train images computational time rose exponentially while not necessarily improving results)

Our goal was to make this threshold adaptive to data, not domain-specific. Otsu's method to determine optimal threshold separating foreground (crack) from background by maximizing between-class variance in the training set predictions.

Test phase

Input: Test images (448×448 RGB)

Two-stage Decision:

1. **Classification Gate:** If classifier confidence < confidence threshold → return empty mask
2. **Segmentation:** Otherwise, apply U-Net → probability map
3. **Otsu threshold:** binary mask

Output: Binary segmentation mask

Our Other Ideas

1. We initially tried using vanilla Grad-CAM to generate pseudo-labels, but it struggled with the problem that it highlighted only the thickest, highest-contrast portion of a crack while ignoring the thinner extensions. This led to fragmented, blob-like segmentations that failed to capture the true linear geometry of the cracks. We switched to Grad-CAM++ because its use of pixel-wise weighting and higher-order gradients allows it to account for lower-intensity features across the entire feature map. Unlike Grad-CAM's global average, Grad-CAM++ more effectively captures the full length and connectivity of thin cracks, providing the object completeness necessary for accurate weakly supervised semantic segmentation.
2. As described earlier, initially we used ResNet without dilation however the problem was that the created masks were very vague.
3. For most of the development, we used ResNet18 in the first stage because the training time was significantly shorter (time for 1 epoch: ~7 min vs ~70 min on one of our devices). However, we were curious how the results would change if we used ResNet34. That is why in the final day we adjusted our best model to use ResNet34 instead of ResNet18. It turned out that the metrics (IOT and Dice) were increased by ~0.03. That is why it can be beneficial to adjust the architecture to use even deeper ResNet like ResNet50, but we don't have enough computing power to test it.
4. We were also intrigued by HRNet (High-Resolution Net). We thought that it might be beneficial to use it instead of ResNet as its last layer was 112x112. We hoped that it would solve the problem with vagueness even better than ResNet with dilation. However, it turned out that replacing ResNet (ResNet18 in that time) with the lightest implementation of HRNet from timm library lowered the results by roughly 0.04. That is why we rejected this idea and came back to ResNet.
5. Somewhere in the middle of the project, we also tried to utilize Conditional Random Field (CRF). It is a statistical model used to refine segmentation results by ensuring that neighboring pixels with similar properties (like color and proximity) receive the same label. Unfortunately, it had almost no impact on the results so we decided not to use it. We still don't know why as it should have led to better results. We wanted to study the cause and add it to our final model but we lacked time so it is a potential area of improvement for the future

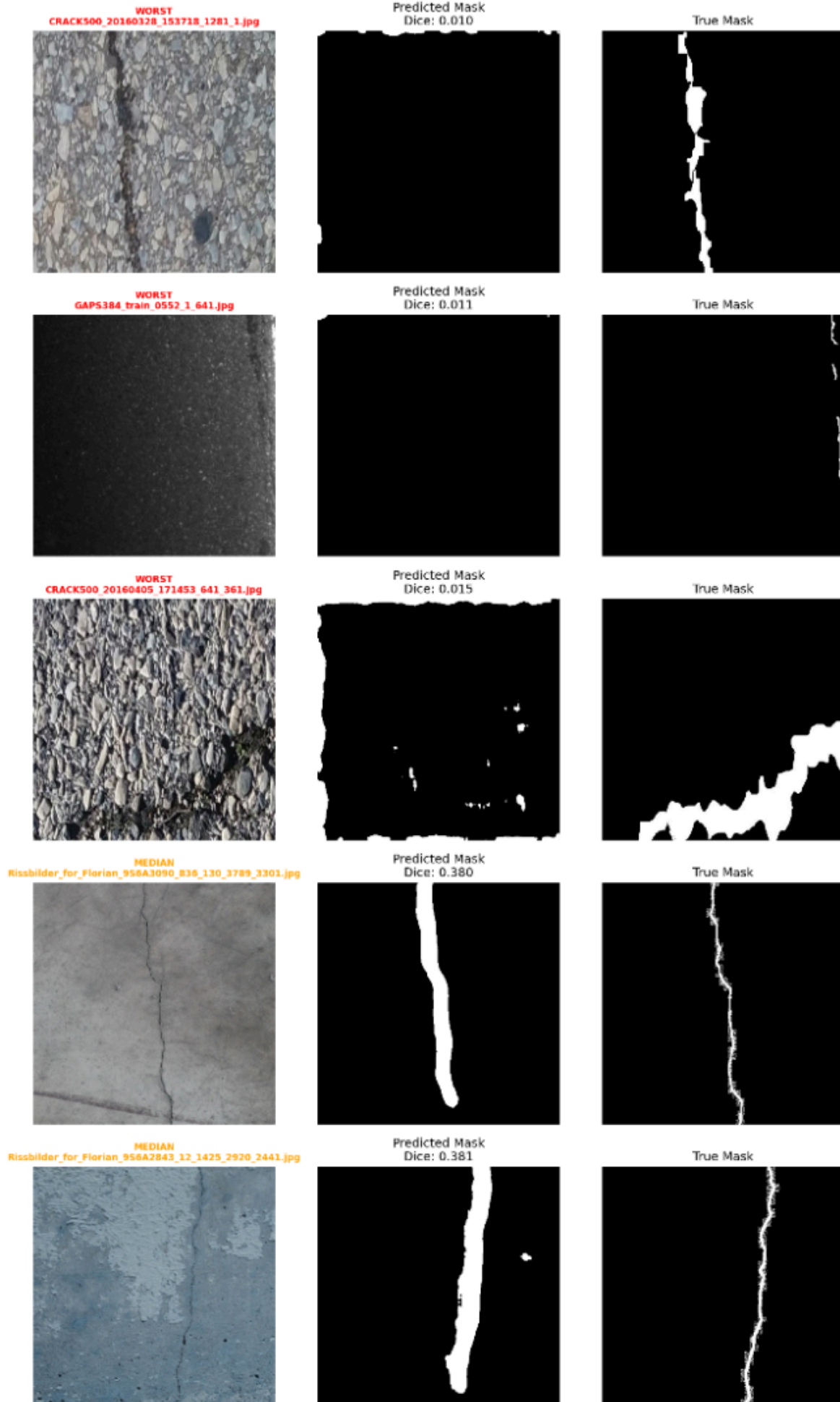
Finally, a screenshot from our repo gives an idea that we have tested several approaches (branches names suggest some of them), some approaches didn't make it to be a branch :) Still we have a lot of ideas but our main constraint is time, especially computational-wise, results submitted to Kaggle were only after 2 epochs, and still it took around two hours to run. With more epochs we would assume better and more stable results, here with 1 or 2 epochs run, our results deviated a bit when testing.

main	yesterday	Default	
ResNet34_with_dilation-GradCAM++-UNet	4 hours ago	0 2	
ResNet_with_Dilation-GradCAM++-UNet	yesterday	2 1	
thresholds-tuning	last week	11 0	#7
CRF	last week	18 1	
mvp	last week	20 1	
normalization	last week	20 1	
resnet-gradcam-unet	2 weeks ago	28 0	#1
Grad-CAM++	2 weeks ago	39 1	
li-threshold	2 weeks ago	38 0	

Repository available at: <https://github.com/jakubradziejewski/crack-detection>

Below visualization of three worst ($\text{dice} \geq 0.1$), median and best ($\text{dice} < 1$), made by our model in the final run.

3 Worst, Median, and Best Predictions



MEDIAN
Rissbilder_for_Florian_956A2867_114_180_3373_2642.jpg



Predicted Mask
Dice: 0.381



True Mask



BEST
Sylvie_Chambon_038.jpg



Predicted Mask
Dice: 0.786



True Mask



BEST
Sylvie_Chambon_039.jpg



Predicted Mask
Dice: 0.806



True Mask



BEST
Sylvie_Chambon_069.jpg



Predicted Mask
Dice: 0.823



True Mask

